

Deduce, Propose, Correct: Probability-Accountable LLM Guidance for Typed Program Inference

Tri Nguyen

Thanh-Dat Nguyen

Harvard University

Basis Research Institute

datnguyen@seas.harvard.edu

Abstract

Large language models can guide program synthesis, but their practical value is difficult to separate from symbolic pruning, test execution, and selection. Moreover, sequential Monte Carlo requires the probability of each proposal, whereas free-form code generation exposes no tractable probability for a canonical program. We introduce DPC, a controlled framework for typed programming by example in a bounded functional language. Type-directed generalization proposes program skeletons; sound deduction refutes inconsistent skeletons and derives examples for typed holes; and a language model scores finite canonical choices. A defensive mixture of language-model energies, deduction, and uniform support yields an explicit proposal that includes family, hole, construction-trace, and cloning probabilities and can therefore be importance-corrected. Exact enumeration supplies an oracle for finite-target diagnostics, while a separate lazy path measures online program evaluations. In an exploratory one-seed grid spanning 3B, 7B, 14B, and 32B Qwen2.5-Coder checkpoints, Qwen-only search failed both tasks at every size, whereas the Qwen-plus-deduction arm found held-out-correct programs in every cell. Checkpoint size did not separate discovery outcomes, and cache disposition prevents a checkpoint-efficiency comparison. Under these conditions, success is attributable to defensive deduction; it does not establish independent Qwen value, checkpoint scaling, faster synthesis, or an SMC/resampling benefit.

Keywords: program synthesis, programming by example, large language models, sequential Monte Carlo, neuro-symbolic inference

1 Introduction

Programming by example (PBE) asks for a program that maps each supplied input to its paired output. The specification is accessible, but it is incomplete: many programs may fit a small example set, and the space of typed recursive programs grows combinatorially. Classical systems address this tension with domain languages, structural hypotheses, deduction, and an Occam-style search order (Gulwani, 2011; Feser et al., 2015). Recent systems instead use learned policies or large language models (LLMs) to prioritize plausible code (Kalyan et al., 2018; Barke et al., 2024; Li et al., 2024).

The empirical question “did the LLM help?” is nevertheless easy to answer incorrectly. A system can attribute success to an LLM even when a type checker, deduction rule, exhaustive catalog construction, or post-hoc selector already identified the solution. An LLM can also assign high probability to a compact but semantically wrong serialization. Reporting

only the best passing sample hides both effects. A useful scientific instrument must isolate the LLM’s proposal from the symbolic support, target score, and selection mechanism.

Sequential Monte Carlo (SMC) provides a natural population view of synthesis: programs are particles, semantic fit defines a potential, and resampling moves computation toward promising regions. This view appears in neural program synthesis (Ellis et al., 2019) and, more recently, in ModelSMC’s inference over executable scientific models (Wahl et al., 2026). Its probabilistic interpretation, however, depends on the proposal probability $q(e' | e)$. For free-form LLM output, many token strings may denote the same abstract syntax tree (AST), invalid generations induce fallback mass, and common APIs do not return the probability of the canonical AST actually consumed by the search.

We study a narrower setting in which this proposal-density gap can be closed. The program language, constants, structural cost, and hole catalogs are bounded. The LLM does not generate arbitrary code; it assigns teacher-forced energies to a complete finite set of canonical family or hole choices. The application normalizes these energies, combines them with a sound deduction guide and a uniform floor, and records the probability of every selected construction step. This produces an evaluable proposal without pretending that it is the free-form probability of an AST.

Our contributions are: (i) a type-directed factorized construction space that combines structural generalization, sound refutation, and derived hole examples; (ii) a canonical finite LLM-energy proposal with explicit family, sequential-hole, construction-trace, and clone accounting; (iii) an importance-corrected Feynman–Kac target whose materialized finite controls can be checked against exact enumeration; and (iv) an evaluation protocol that separates uniform, deduction-only, LLM-only, and combined proposals and preserves negative results. We do not claim that the current bounded language is a general synthesizer, that finite- N particles are a calibrated posterior, or that full enumeration is faster than search.

2 Background and Problem Setting

Let $\mathcal{D} = \{(x_j, y_j)\}_{j=1}^M$ be examples with an inferred signature $\tau_{\text{in}} \rightarrow \tau_{\text{out}}$. A program e is exact on $\mathcal{D}$ when $e(x_j) = y_j$ for every j . We work with a typed functional DSL containing integer and Boolean expressions, lists, conditionals, `map`, and right folds. A structural cost $C(e)$ prefers compact combinator programs, following the minimum-cost perspective of Lambda² (Feser et al., 2015). Exactness on $\mathcal{D}$ is not a proof of the unknown intended semantics, so evaluation also uses held-out inputs generated before confirmatory runs.

For retained finite construction-trace support $\mathcal{E}_{\mathcal{D}}$, we define a data-conditional base measure. A trace $z = (h, c_{1:K})$ records a family and one choice for each typed hole, and assembles to program $e(z)$. If $h(z) \in \mathcal{H}$ denotes its family, then

$$p_0(z | \mathcal{D}) = \frac{1}{|\mathcal{H}| \sum_{u \in \mathcal{E}_{h(z)}} \exp[-\lambda_C C(e(u))]} \exp[-\lambda_C C(e(z))]. \quad (1)$$

The equal family mass prevents a large catalog from dominating solely through cardinality. Because deduction and bounded-catalog policy depend on $\mathcal{D}$, p_0 is a conditional Occam measure, not an unconditional Bayesian prior.

At inverse temperature β , the finite Gibbs target is

$$\tilde{\pi}_\beta(z) = \mathbf{1}[z \in \mathcal{E}_\mathcal{D}] p_0(z \mid \mathcal{D}) \exp[-\beta \lambda_L L_\mathcal{D}(e(z))], \quad (2)$$

where $L_\mathcal{D}$ is a deterministic soft execution loss. This is a declared finite target for controlled inference. It is not a posterior over all programs and the loss exponential is not asserted to be a real-world likelihood.

3 Typed Generalization and Deduction

Generalization maps the inferred signature and structural relations in the examples to typed hypotheses. For list-to-list tasks these include a direct expression, a map, a generic fold, and factorized filter-map folds. Each hypothesis is a program skeleton with typed holes. For example,

$$\text{foldr}(\lambda i, a. \text{ if } p(i) \text{ then } g(i) :: a \text{ else } a, [], x) \quad (3)$$

has holes $p : \text{Int} \rightarrow \text{Bool}$ and $g : \text{Int} \rightarrow \text{Int}$.

Deduction has two roles. Refutation removes a skeleton only when an example contradicts an invariant; for instance, **map** is impossible when input and output lengths differ. Example inference transfers top-level evidence to a hole: singleton and aligned list examples produce facts such as $p(-3) = \text{false}$ or $g(2) = 4$. Ambiguous alignments do not generate facts. Deduction therefore changes proposal guidance while preserving all candidates not ruled out by a sound invariant.

Each surviving hole receives a complete bounded catalog of well-typed canonical ASTs. Generic holes are enumerated by structural cost; declared factorized skeletons use explicit finite catalogs. The online state is the family-plus-hole construction trace, so two traces that assemble to the same AST remain distinct latent states. The materialized canonical-program control may instead apply an explicit owner rule and records the discarded-alias count. We compare its oracle metrics with online traces only when the construction map is injective on the reported support; otherwise the two modes define different targets.

4 Canonical LLM and Deduction Proposals

At a construction node A with finite choices $\mathcal{C}_A$, Qwen receives a common prompt containing the task, typed hole, sound deductions, current ancestor, and previous fillings. For each canonical choice c , vLLM returns teacher-forced prompt-token log probabilities for the complete concatenated text. These values define an application-level finite energy; they are not interpreted as a continuation probability or the probability of a free-form AST. We archive the prompt prefix, candidate strings, token identifiers, per-token scores, energy semantics, and declared model and tokenizer revision labels so that the Qwen categorical can be replayed. Specifically, if $\ell_\theta(A, c)$ is the sum of the scored conditional token log probabilities for the complete concatenated prompt and $n(A, c)$ is its number of scored positions, the implementation supports $s_\theta(A, c) = \ell_\theta(A, c)$ and $\bar{s}_\theta(A, c) = \ell_\theta(A, c)/n(A, c)$. Both are full-prompt energies; the mean is not a candidate-suffix likelihood because no separate tokenizer boundary between the textual prefix and candidate is assumed.

Let $s_\theta(A, c)$ be the declared Qwen energy and let $r_A(c)$ be the exact subtree marginal of an Occam-weighted deduction guide. The categorical used for both sampling and correction

is

$$q_A(c) = (1 - \varepsilon) \left[(1 - \rho) \text{softmax} \left(\frac{s_\theta(A, \cdot)}{\tau} \right)_c + \rho r_A(c) \right] + \frac{\varepsilon}{|\mathcal{C}_A|}. \quad (4)$$

The uniform term gives every declared choice positive mass. Mixing normalized distributions, rather than adding token scores to logical penalties, prevents long JSON serializations from overwhelming deduction merely through cumulative log probability.

A program trace z selects one family and an ordered sequence of hole fillings:

$$q_{\text{construct}}(z \mid a, \mathcal{D}) = q_H(h \mid a, \mathcal{D}) \prod_k q_{A_k}(c_k \mid a, h, c_{<k}, \mathcal{D}). \quad (5)$$

When only one family survives, $q_H = 1$ and no provider call is made. Identical requests created by resampling are scored once and fanned out without changing the distribution.

With clone probability $\alpha < 1$, the transition on construction traces is

$$Q_\alpha(z' \mid z) = \alpha \mathbf{1}[z' = z] + (1 - \alpha) q_{\text{construct}}(z' \mid z, \mathcal{D}). \quad (6)$$

For $z' = z$, both routes are combined with a log-sum-exp. Provider errors abort the accountable mode because an unmodeled ancestor fallback would add unknown mass. A run-wide score budget is reserved before each request wave.

5 Importance-Corrected SMC

For stages $t = 1, \dots, T$, define the path target $\gamma_t(z_{1:t}) = \prod_{s=1}^t \tilde{\pi}_{\beta_s}(z_s)$ and transition $M_t = Q_\alpha$. The incremental potential is

$$G_t(z_{t-1}, z_t) = \frac{\tilde{\pi}_{\beta_t}(z_t)}{Q_\alpha(z_t \mid z_{t-1}, \mathcal{D})}. \quad (7)$$

Thus $M_t G_t = \tilde{\pi}_{\beta_t}$ and the normalized terminal-state marginal is the declared finite target. If resampling occurs, ancestor weights reset to $1/N$; otherwise the previous normalized weights remain the base.

The product of stage normalizers belongs to the path target, not merely the final Gibbs target. Exact enumeration on small supports supplies the reference $\sum_t \log Z_{\beta_t}$ and the final target distribution. We report total-variation distance, exact-program mass, mean loss and cost, effective sample size (ESS), and normalizer error. These diagnostics measure a finite particle approximation; they do not imply that a small particle set is itself calibrated.

The implementation exposes two execution boundaries. The reference mode materializes and evaluates every complete bounded program so that target functionals are known exactly. The online mode first samples initial traces and then proposal traces, assembling and evaluating only previously unvisited samples. It reports search cost but cannot report exact TV or normalizer error unless a matching external oracle is run separately. Oracle comparisons require the same trace semantics; any online-oracle comparison must first verify that cross-family aliases do not change the state space. This separation prevents exhaustive support construction from being counted as an LLM or SMC synthesis success.

Table 1: Controlled enumerable tasks; counts are fixed regression-test targets.

Task	Examples	Support	Exact states	Max choices/path
Integer sign predicate	5	82	1	82
Map increment	4	16,012	3	392
Fold sum	5	22,914	46	316
Positive filter	4	24,646	144	518
Bounded square	14	36,198	2	663
Signed window	14	132,198	4	823

6 Experimental Protocol

Our planned factorial comparison has four proposal arms: uniform (U), deduction plus uniform (D), Qwen (Q), and Qwen plus deduction (Q+D). All arms share type inference, sound refutation, finite catalogs, target, seeds, particles, iterations, and proposal-score caps. Paired contrasts D−U, Q−U, Q+D−D, and Q+D−Q isolate the soft guides. The current protocol does not include a separate uncorrected arm or a one-stage versus multi-stage re-sampling ablation; either would require an additional pre-registered matrix. The exploratory Gate-2 slice reported below contains only the paired Q and Q+D arms and therefore cannot estimate a deduction-only or resampling effect.

Each run archives exact training consistency, held-out semantic correctness, per-stage newly and cumulatively realized traces, finite choices scored, cache hits and misses, provider-scored token positions and wait, wall time, ESS, resampling, final particles, and failures. Figure 2 separates new provider work from cached score reuse. The reported grid uses the frozen exploratory transport-amendment protocol with SHA-256 56c590864d456f884c82bf62ada3c11c1c2504d21b021e650bc323007553fc60. After an HTTP 524 on the 32B endpoint, this amendment changed only candidate transport batch size from 128 to 32; all reported cells share the amended hash, and superseded preflight cells are excluded. The seed, not a particle, is the independent unit. Because this slice has one seed and no implemented confidence intervals, hypothesis tests, or multiplicity adjustment, it is exploratory and cannot support a population or checkpoint-size claim.

Table 1 summarizes the current controlled benchmark. The two no-solution support diagnostics are excluded from success rates and reported separately. These tasks exercise scalar expressions, map, fold, filter-map, and signed piecewise mapping. They remain a small synthetic DSL benchmark; external PBE or SyGuS-derived tasks are required before making a broad synthesis claim.

7 Exploratory Gate-2 Findings

Figure 1 reports the complete exploratory Gate-2 slice: four Qwen2.5-Coder checkpoints, two tasks, paired Q and Q+D arms, seed 101, four particles, and one SMC stage. Q failed exact training discovery in all eight cells; its held-out accuracy was 0 on map increment and 1/3 on signed window. Q+D found a training-exact program with held-out accuracy 1.0 in all eight cells. Thus, within this fixed seed and grid, adding defensive deduction changed every discovery outcome. These 16 cells contain only one independent seed—the

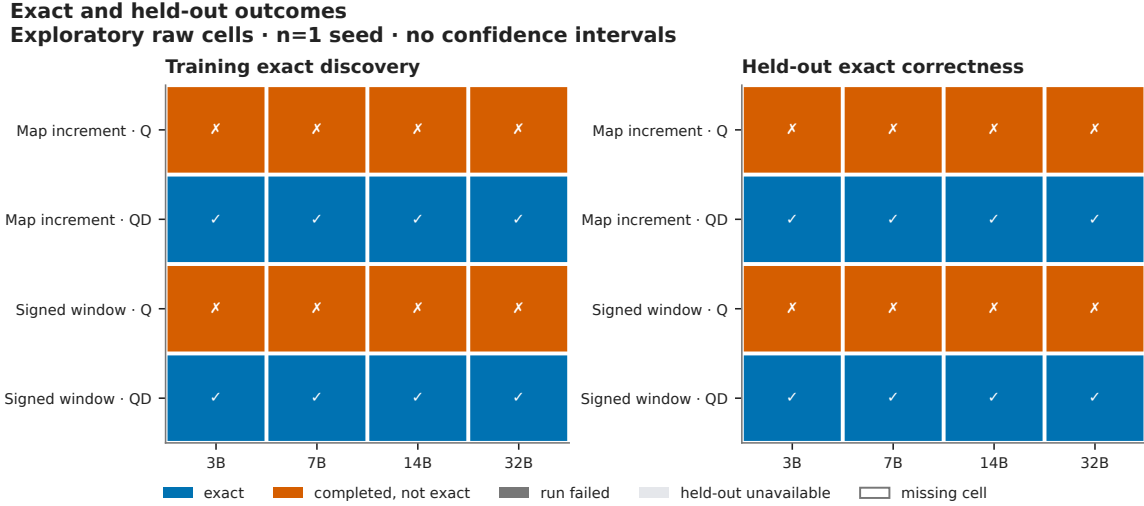


Figure 1: Training and held-out outcomes for the complete exploratory Gate-2 grid under one amended protocol hash. Each tile is a raw one-seed cell; crosses and checks are not estimates with confidence intervals.

four particles are not replicates—so they neither estimate success rates nor establish a general model effect.

Checkpoint size did not separate outcomes: the 3B, 7B, 14B, and 32B columns show the same Q failure and Q+D success pattern on both tasks. Because the paired arms differ by the deduction mixture, this pattern shows that defensive deduction dominated discovery under these tested conditions. It does not show that Qwen added value independently of deduction: Gate 2 contains no D arm, and one seed cannot estimate a Qwen-by-size interaction. Nor does a one-stage grid identify a benefit from SMC resampling.

The outcome invariance does not imply equal computational behavior. The provider-work panel is an accounting diagnostic, not a checkpoint timing comparison: the 32B map cells are fully warm replays of superseded preflight scores, the 32B signed-window cells reuse part of that cache, and other cells have different cold or mixed dispositions. Cached scores are replayed evidence, not new model work, so raw waits across these cells cannot estimate checkpoint efficiency. The grid used mean full-prompt energies; those energies remain application-level scores, not candidate-suffix or AST probabilities.

Provider work and cache disposition

Exploratory raw seed cells · cached reuse is not new provider work

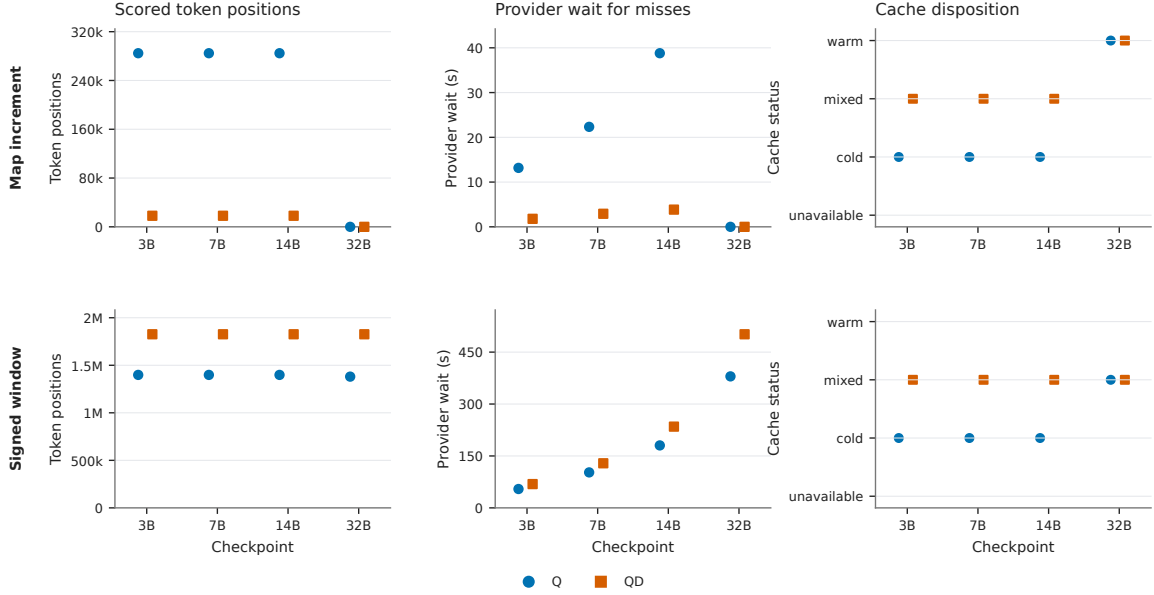


Figure 2: Provider work for the complete exploratory Gate-2 grid. Token positions and provider wait count cache misses only; cached reuse is shown separately and is not new provider work. The plot reports raw cells, not a speed comparison or a checkpoint-scaling estimate.

8 Related Work

Lambda² combines type-aware inductive generalization, deductive refutation, derived examples, and best-first enumeration for recursive data structures (Feser et al., 2015). Neural-guided deductive search and DreamCoder learn distributions that prioritize symbolic search (Kalyan et al., 2018; Ellis et al., 2021). HYSYNTH derives a context-free surrogate from LLM completions, and LLM-guided enumeration iteratively exchanges search state with a model (Barke et al., 2024; Li et al., 2024). Our narrower focus is the exact probability accounting of a canonical factorized energy proposal and its separation from symbolic deduction and target scoring.

Write–Execute–Assess combines learned program policies with SMC (Ellis et al., 2019); ModelSMC represents executable models as particles and uses LLM refinement (Wahl et al., 2026). SMC also steers token generation under probabilistic-program constraints (Lew et al., 2023; Zhao et al., 2024). We do not claim the first use of SMC or LLMs in synthesis. The contribution is a finite setting in which family and hole proposal terms, defensive mixtures, and clone routes are all evaluable and can be checked against an exact target.

9 Limitations and Broader Impact

The DSL and current tasks are deliberately small. Specialized filter-map catalogs encode useful structural knowledge, support depends on observed data, and multiple training-exact programs may disagree on unseen inputs. Exact enumeration scales poorly and is an oracle, not the online algorithm. The semantic core is extensively tested but not formally verified. Model and tokenizer revisions can change Qwen energies, and full-prompt scoring has a serialization-length confound. These boundaries rule out claims of general program synthesis, formal correctness, or a posterior over unbounded source code.

The lazy implementation avoids executing complete programs that are never sampled, but the one-seed grid does not establish lower wall time or fewer total resources because symbolic catalog construction and Qwen scoring also have cost. The method can also amplify biases encoded in prompts or the model. A uniform floor preserves mathematical support, not social fairness or semantic safety. Generated programs must remain sandboxed, resource-bounded, and reviewed before deployment. We publish failed runs and complete score ledgers so that readers can audit adverse model preferences rather than only successful outputs.

10 Conclusion

LLM-guided synthesis needs more than a plausible generated program: it needs a clear account of what the model proposed, what symbolic reasoning removed or favored, and how selection changed the resulting measure. DPC turns typed PBE into a finite construction problem whose LLM energies, deduction guide, uniform support, and importance correction are separately inspectable. The exploratory grid shows why that decomposition matters: Q alone failed every reported cell, whereas the defensive Q+D mixture succeeded. This finding attributes discovery to deduction under the tested conditions, not to Qwen alone, checkpoint size, or SMC resampling. Repeated-seed benchmarks with the D control must determine whether and where the LLM adds value.

Acknowledgments and Disclosure of Funding

This draft received no declared external funding. Compute was run on a rented RunPod NVIDIA A100 instance. The final submission must replace this statement with complete funding, donated-compute, competing-interest, and author disclosures.

Appendix A. Proposal Normalization

For every node A , Equation 4 is a convex combination of three normalized categorical distributions and therefore sums to one. Because $\varepsilon > 0$, each choice has mass at least $\varepsilon/|C_A|$. Products over the finite construction trie are normalized by induction on trie depth. Equation 6 is a mixture of a point mass and this normalized construction distribution, so it is also normalized and has full support over the declared finite trace space.

Appendix B. Claim Boundary for Exact Enumeration

Reference enumeration discovers whether exact programs exist while computing the oracle. Consequently, an oracle-backed run demonstrates probability accounting and estimator diagnostics, not faster synthesis. The Gate-2 outcome grid reports lazy search-success endpoints and no oracle TV or normalizer metrics. Any oracle comparison must remain external to that search, and both modes must count or label their realized traces separately. An exact state first observed during reference construction is never credited to Qwen or SMC discovery.

References

- Shraddha Barke, Emmanuel Anaya Gonzalez, Saketh Ram Kasibatla, Taylor Berg-Kirkpatrick, and Nadia Polikarpova. HYSYNTH: Context-free LLM approximation for guiding program synthesis. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL <https://openreview.net/forum?id=5jt0ZSA6Co>.
- Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Joshua Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1906.04604>.
- Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 835–850. ACM, 2021. doi: 10.1145/3453483.3454080.
- John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 229–239. ACM, 2015. doi: 10.1145/2737924.2737977.
- Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 317–330. ACM, 2011. doi: 10.1145/1926385.1926423.
- Ashwin Kalyan, Abhishek Mohta, Oleksandr Polozov, Dhruv Batra, Prateek Jain, and Sumit Gulwani. Neural-guided deductive search for real-time program synthesis from examples. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rywDjg-RW>.
- Alexander K. Lew, Zhi-Xuan Tan, Gabriel Grand, and Vikash K. Mansinghka. Sequential monte carlo steering of large language models using probabilistic programs. *arXiv preprint arXiv:2306.03081*, 2023. doi: 10.48550/arXiv.2306.03081.
- Yixuan Li, Julian Parsert, and Elizabeth Polgreen. Guiding enumerative program synthesis with large language models. *arXiv preprint arXiv:2403.03997*, 2024. doi: 10.48550/arXiv.2403.03997.

Stefan Wahl, Raphaela Schenk, Ali Farnoud, Jakob H. Macke, and Daniel Gedon. A probabilistic framework for LLM-based model discovery. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2602.18266>.

Stephen Zhao, Rob Brekelmans, Alireza Makhzani, and Roger Grosse. Probabilistic inference in language models via twisted sequential monte carlo. *arXiv preprint arXiv:2404.17546*, 2024. doi: 10.48550/arXiv.2404.17546.